

嵌入式操作系统

Embedded Operating System

信息与软件工程学院

桑楠

sn@uestc.edu.cn

2026年4月

第六部分

内核移植

核心内容

- 基本思想
- HAL层移植
- 项目移植

基本思想

基本概念

- **移植**：将程序代码从一种运行环境转移到另一种运行环境
- **内核移植**：将内核代码从**一种硬件平台**转移到**另一种硬件平台**上运行。
- **内核移植**：将已在某一特定CPU(或SOC芯片)上运行的RTOS内核在另一CPU(或SOC芯片)上运行起来
- **内核移植**：移植工作大部分是和硬件相关，需要针对具体CPU或芯片进行有区别的代码编写
- **为何需要移植**：
 - 基本内核庞大，不符合应用需要，如Linux
 - 厂家只提供了一种环境的Demo

基本概念 (2)

- 为何需要移植：
 - 应用环境以前不支持某种RTOS
 - 从裸板开始
 - 硬件可裁剪性：对不同应用，工程师们设计的硬件电路会有所不同，导致原代码可能无法正确运行 → 必须结合自己的硬件电路，对已有的内核代码进行修改移植。
- 移植的内容：
 - 简单过程：内核配置、内核编译和内核下载
 - 通用过程：硬件抽象层HAL移植，项目移植

Linux基本移植过程

- 嵌入式Linux移植：简单过程
- 准备工作：
 - 确定硬件体系结构：如ARM9
 - 下载内核源码：如linux-2.6.37.tar.bz2
 - 准备工具：如对应版本的gcc
- 准备内核：获得将安装到硬件上的Linux内核
 - 解压源代码：如 `tar -jxvf linux-2.6.37.tar.bz2`
 - 修改Makefile：不同应用需要有自己的makefile

Linux基本移植过程 (2)

- **配置内核**：在已有配置文件基础上，修改得到新配置文件
- **编译内核**：根据配置中选中的内容，开始编译内核
- **初步移植**：将编译好的内核安装到硬件上
 - **准备**：开发板上有u-boot或bootloader
 - **设置下载地址**：如0x30008000 —— 启动位置
 - **下载、测试**：
- **进一步移植**：修改内核代码、添加驱动、重新编译下载

基于参考板的移植过程

- 准备工作
 - 选择参考板
 - 编译测试参考板上的内核：如Linux
 - 分析参考板上的BSP代码
- 开发板内核移植
 - 添加开发板初始项：
 - 移植开发板驱动程序：SDRAM、Flash、以太网口等
- 内核发布：已经移植完成

通用移植过程

- **HAL移植**：针对不同目标板改写相关代码
 - 不同开发板的硬件资源不一样的
 - 不同架构处理器的指令集不一样
 - 相同架构处理器的不同系列产品，寄存器资源也不同
- **项目移植**：处理规则不一致的问题
 - 不同开发环境的编译器、汇编器、链接器不一样
 - 同样的C代码，也存在兼容性问题
 - 需要针对不同开发环境编写专门的汇编代码

HAL移植

与硬件相关的代码

- **启动**：启动过程、...
- **中断**：中断环境保存/恢复、中断屏蔽、开/关中断、时钟中断等操作
- **任务切换**：任务环境保存/恢复
- **内存**：不同CPU支持的内存不一致，不同RTOS管理方式也不一致
- **驱动程序**：设备的控制接口等

aCoral的HAL移植

- **启动接口**：从start.s开始到C语言入口函数acoral_start，其中

CPU相关部分的修改

链接文件aCoral.lds

```
ENTRY(__ENTRY)
MEMORY
{
  RAM (wx) : org = 0x30000000, len = 64M
}
SECTIONS
{
  .text :
  {
    text_start = .;
    * (.text)
    * (.init.text)
    * (.rodata*)
  } >RAM
  .....
}
```

起始地
址

aCoral的HAL移植 (2)

- **中断接口**：hal_int_s.s文件中的各接口，包括
HAL_INTR_ENTRY、HAL_INTR_INIT、HAL_INTR_SPECIAL、

HAL_INTR_ENTRY:

@中断入口

stmfd sp!, {r0-r12,lr}

@保护通用寄存器及PC

mrs r1, spsr

stmfd sp!, {r1}

@保护spsr，以支持中断嵌套

msr cpsr_c, #SVCMODE|NOIRQ

@进入SVCMODE，以便允许中断嵌套

.....

ldmfd sp!, {r0}

@spsr->r0

msr spsr_cxsf, r0

@恢复spsr

ldmfd sp!, {r0-r12,lr}

subs pc, lr, #4

@中断被重新打开

aCoral的HAL移植 (3)

- **任务相关接口**：文件hal_thread_s.s 中的各接口，包括
HAL_SWITCH_TO、HAL_START_OS、HAL_CONTEXT_SWITCH、

HAL_CONTEXT_SWITCH:

stmfd sp!,{lr}

@保存PC

stmfd sp!,{r0-r12,lr}

@保存寄存器LR，及r0-r12

mrs r4,CPSR

stmfd sp!,{r4}

@保存cpsr

str sp,[r0]

@保存旧上下文栈指针到旧的任务prev->stack

ldr sp,[r1]

@取得新上下文指针

ldmfd sp!,{r0}

msr cpsr,r0

@恢复新cpsr(不用spsr, sys,user模式没有SPSR)

ldmfd sp!,{r0-r12,lr,pc}

@恢复寄存器,

aCoral的HAL移植 (4)

- **时间相关接口**：以Ticks为基础的时钟中断处理，主要有
HAL_TICKS_INTR、HAL_TICKS_INIT

```
void hal_Ticks_init(){  
    rTCON = rTCON & (~0xf);  
    rTCFG0 &= 0xFFFF00;  
    rTCFG0 |= 0xF9;           // prescaler等于249  
    rTCFG1 &= ~0x0000F;  
    rTCFG1 |= 0x2;           //设置定时器4的时钟频率为25kHz  
    rTCNTB0 = PCLK  
    rTCON = rTCON & (~0xf) | 0x02; //手动更新Timer0  
    rTCON = rTCON & (~0xf) | 0x09; //启动定时器  
}
```

aCoral的HAL移植 (5)

// hal_mem.h, Created on: 2010-3-7: S3C2410

```
#ifndef HAL_MEM_H_
```

```
    #define HAL_MEM_H_
```

```
#include<type.h>
```

```
extern acoral_u32 heap_start[];
```

```
extern acoral_u32 heap_end[];
```

```
#define HAL_HEAP_START heap_start
```

```
#define HAL_HEAP_END heap_end
```

```
#define HAL_MEM_INIT() hal_mem_init()
```

```
void hal_mmu_setmtt(int vaddrStart,int vaddrEnd,int paddrStart,int attr);
```

```
#endif /* HAL_MEM_H_ */
```

CPU
确定

aCoral的HAL移植 (6)

- **多核相关接口**：针对多核嵌入式处理器，主要涉及
HAL_CORE_CPU_INIT（**主核初始化**）、
HAL_FOLLOW_CPU_INIT（**次核初始化**）、HAL_START_CPU
（**激活某一CPU**）、等

aCoral的HAL移植 (6)

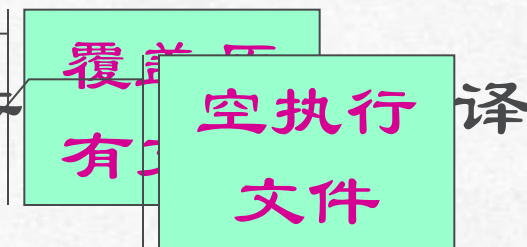
- **开发板相关接口**：开发板上一些设备控制器初始化，主要是

HAL_BOARD_INIT

```
void hal_board_init(void){  
    hal_cpu_init();           //若Bootloader已做，这里就忽略  
    hal_io_init();            //预留外设驱动的初始化，  
                               //也可以在OS启动后做  
}
```

移植时必须关注的问题

- 规范化：
- 启动文件：必须有start名字的启动文件
器等变化
- 头文件：针对可能移植的接口，需要定义两个头文件——需要移植的、不必移植的
- 命名规范：对每类需要移植的接口，规范命名，如硬件相关acoral中采用HAL_XXX



移植示例：统一入口处理

__ENTRY: @ARM S3C24xx的中断入口

b ResetHandler

.....

b HandleIRQ @handler for IRQ interrupt

b HandleFIQ @handler for FIQ interrupt

.....

HandleFIQ:

ldr pc,=acoral_start

HandleIRQ:

ldr pc,=HAL_INTR_ENTRY

.....

移植示例：时钟

```
#include "acoral.h"    // S3C2440

void hal_Ticks_init(){
    rTCON = rTCON & (~0xf);
    rTCFG0 &= 0xFFFFF00;
    rTCFG0 |= 0xF9;
    rTCFG1 &= ~0x0000F;
    rTCFG1 |= 0x2;
    rTCNTB0 = PCLK / (8 * (249 + 1) *
        ACORAL_TickS_PER_SEC);
    rTCON = rTCON & (~0xf) | 0x02;
    rTCON = rTCON & (~0xf) | 0x09;
}
```

```
//S3C2410

void hal_ticks_init(){
    rTCON = rTCON & (~0xf);
    rTCFG0 &= 0xFFFFF00;
    rTCFG0 |= 0xF9;
    rTCFG1 &= ~0x0000F;
    rTCFG1 |= 0x2;
    rTCNTB0 = PCLK / (8 * (249 + 1) *
        ACORAL_TICKS_PER_SEC);
    rTCON = rTCON & (~0xf) | 0x02;
    rTCON = rTCON & (~0xf) | 0x09;
}
```

项目移植

项目移植的类别

- **同OS下软件项目移植**：如将多个基于MFC/Win32/MMC1.2的对话框移植到基于C#/.NET Framework 2.0/MMC2.1版本的对话框
- **不同OS间软件项目移植**：如VS2008项目移植到I
- **针对不同硬件平台的软件项目移植**：如将Android手机上的游戏移植到Iphone手机上
- **移植的基本限制**：相互之间比较类似的需求/项目

MS管理控制平台

针对不同开发板

aCoral: 现有环境

- **HAL环境**: 包括如下子目录: board、mk、ads、include、src等
 - **board**: 可以包含S3C2410、S3C2440、...; 即使是2410也可能有多种开发板
 - **ads**: ARM开发环境, 其中包含相关配置
- **Drivers环境**: 包含如下子目录: S3C2440、S3C2410、LPC2200、STM3210、src、include等

为何只有这两部分?

生成对应开发板的项目

- **创建一个项目环境Proj**：类似于利用Eclipse创建项目环境；但对许多嵌入式项目，相关的文件夹（如include、lib、src、drivers、...）等必须手工创建
- **芯片相关的文件移植**：针对S3C2410，需要将HARDWARE/S3C2410相关文件夹移到Proj环境中（**拷贝**）
- **驱动相关的文件移植**：针对S3C2410，需要将drivers下S3C2410相关文件夹移到Proj环境中

是否完成了？

调整/修改项目内容

- **驱动文件的变更**：保留相同外设的驱动（其中可**这部分较难！**），要根据CPU的支持情况改写部分代码），增加原环境中不存在的外设驱动
- **内核文件的变更**：参照前文——**可能需要配置相关环境文件**
- **示例**：基于无线网络的信息显示系统——从Intel环境移植到ARM环境

编译运行内核

- **编译内核**：用Ubuntu的GCC编译移植过来的aCoral内核，其中包含利用aCoral的简单工具进行相关配置
- **烧写内核**：将编译好的内核烧写到目标板上
 - **连接串口**：minicom
 - **安装烧写工具**：DNW
 - **烧写与运行aCoral**

思考题

1. 什么是内核移植？aCoral的移植工作包括哪些内容？
2. RTOS内核中与硬件相关的代码通常包括哪几个方面？
3. 当ARM9 S3C2410产生复位异常时，软件上需要做什么处理？请结合代码加以描述。



谢谢！